Quantum-Safe Bitcoin Transactions Today Using Quantum Safe Binohash (QSB)

Avihu Mordechai Levy
@avihu28
avihu@starkware.co

April 1, 2026

Abstract

We present a scheme for quantum-safe Bitcoin transactions that requires no changes to the Bitcoin protocol and remains secure even in the presence of Shor's algorithm. Our construction modifies Binohash (Linus, 2025) to replace the signature-size-based proof-of-work puzzle—which relies on ECDSA hardness—with a hash-based puzzle whose security depends only on the pre-image resistance of RIPEMD-160. The resulting scheme, Quantum Safe Binohash (QSB), achieves ∼117-bit second pre-image resistance at an off-chain cost of approximately $40 under the Shor threat model, and roughly half the security bits under Grover's algorithm. A security–cost tradeoff exists: ∼130-bit pre-image resistance is achievable at a higher off-chain cost of approximately $1,000. Both configurations fit within Bitcoin's legacy script constraints of 201 opcodes and 10,000 bytes.

1 Introduction

Bitcoin transactions are not quantum-safe. The primitive that binds the components of a transaction—its inputs, outputs, and spending conditions—is an ECDSA signature (pre-Taproot) or a Schnorr signature (post-Taproot), neither of which is post-quantum secure. This means that even spending conditions which are themselves quantum-safe can be subverted: if the signature scheme securing the transaction is broken, the entire transaction—including its outputs and other inputs—can be forged.

Consider, for example, a UTXO whose spending condition is knowledge of a hash preimage, which is quantum-safe on its own. A spender broadcasts a transaction revealing the preimage. An adversary who can break the signature scheme can take that preimage, construct a different transaction t' spending the same UTXO to an address they control, sign t' with a forged signature, and broadcast it instead.[1]

1.1 Related Work

Two recent works bring us close to the goal of quantum-safe Bitcoin transactions. The first is Heilman's "Signing a Bitcoin Transaction with Lamport Signatures (no OP_CAT)" [1], followed by the work of Robin Linus—Binohash [2]—on which the present scheme is based almost entirely.

While these two works differ significantly in their constructions and security properties, they both rely on a common high-level approach. Inside the Script:

[1] Under Taproot, the situation is strictly worse: every Taproot output is spendable via the key path, meaning knowledge of the private key corresponding to the internal public key (which aggregates the taptweak and the script tree root) suffices to spend it, regardless of the script conditions in the taptree [4]. A quantum adversary capable of computing discrete logarithms could therefore spend any Taproot output directly.

1. **Derive a cryptographically strong identifier** of the spending transaction—call it D.
2. **Verify a Lamport signature** over D.

Since Lamport signatures are post-quantum secure, and they sign a cryptographically strong identifier of the transaction, it is not possible to modify the transaction without producing a new Lamport signature—which the attacker cannot forge, even with quantum computing capabilities.

1.1.1 Remaining Issues

There are two main issues with the existing schemes:

1. Reliance on signature size check. This is the central issue. Both schemes rely on a signature proof-of-work (PoW) puzzle based on signature size [6], verified via `OP_SIZE`. This puzzle assumes a specifically small ECDSA r value for which the discrete logarithm is known. Currently, a well-known minimal value $r_{\min}$ is used. However, an adversary with knowledge of discrete logarithms—as a quantum computer would provide—could compute the smallest possible r value ($r = 1$), breaking the `OP_SIZE` check for all transactions created with the current $r_{\min}$.[2]

2. Sighash flag uncertainty. Since the sighash flag is not directly readable from Script, both works acknowledge potential attacks arising from this limitation. Binohash discusses this in detail [2, Section 7.1], identifying `ANYONECANPAY|NONE` as a potential attack vector that could break the security guarantees.

1.2 Our Contribution

In this work, we modify the Binohash scheme to address both issues identified in Section 1.1.1. We replace the signature-size-based PoW puzzle with a hash-based puzzle that does not depend on the ECDSA r value or on any elliptic curve hardness assumption. Furthermore, since the nonce in our new puzzle is a public key—derived via ECDSA key recovery from a hardcoded signature—we are free to hardcode the appropriate sighash flag (`SIGHASH_ALL`) directly in the locking script, eliminating the sighash flag uncertainty.

2 Background

We assume familiarity with Bitcoin Script, ECDSA signatures, and DER encoding; for a thorough treatment of these topics, see Binohash [2], Sections 2–3. Our scheme builds extensively on the Binohash construction, and we strongly recommend reading that paper prior to this one. While we summarize the relevant components in the following sections, many of the deeper design choices and optimizations—particularly around FindAndDelete, minimum-size signatures, and the SIGHASH_SINGLE bug—are explained more fully there.

2.1 Goals

We aim to construct:

1. A **cryptographically strong identifier** of the spending transaction—with both second pre-image and collision resistance—that is readable within Bitcoin Script.
2. A **Lamport signature** over that identifier, verifiable entirely within Script.

[2] Paradoxically, once $r = 1$ is found, it could be used *going forward* to strengthen these schemes: the PoW puzzle with $r = 1$ would be quantum-safe, since there is no smaller r to exploit. But all transactions created before that point, using the current $r_{\min}$, would be vulnerable.

2.2 Constraints

For multiple reasons—including the reliance on ECDSA (rather than Schnorr), the FindAndDelete mechanism (removed in SegWit), and the SIGHASH_SINGLE bug (absent in SegWit's sighash algorithm)—neither BINOHASH nor our construction can use Taproot or SegWit. We are therefore restricted to legacy (pre-SegWit) script execution, which imposes severe constraints. The two hardest, and therefore most important, are:

1. **Opcode limit.** The number of non-push opcodes in a script may not exceed 201. Every opcode above `OP_16` counts toward this limit, including opcodes in non-executed branches of `OP_IF`/`OP_ELSE` [2, Section 6].

2. **Script size limit.** The total script size may not exceed 10,000 bytes [2, Section 6].

For a full discussion of additional constraints—including stack element size limits, stack depth, and transaction relay considerations—see BINOHASH [2, Section 6].

These constraints are extremely demanding in the context of our two goals: achieving a cryptographically strong transaction identifier requires a high number of security bits, while the Lamport signature imposes a per-bit overhead in both opcodes and script size.

2.3 Threat Model

2.3.1 Quantum Computing Assumptions

Our threat model assumes an adversary with access to a cryptographically relevant quantum computer. We distinguish between the two classes of cryptographic primitives affected:

Elliptic curve cryptography (catastrophic vulnerability). ECDSA and Schnorr signatures rely on the hardness of the elliptic curve discrete logarithm problem (ECDLP). Shor's algorithm [8] solves ECDLP in polynomial time on a quantum computer, rendering these schemes completely insecure. For the secp256k1 curve used in Bitcoin, a sufficiently large quantum computer could recover private keys from public keys, forge arbitrary signatures, and compute discrete logarithms of any curve point. This is a total break: the adversary gains *unlimited* forgery capability, not merely a reduction in work factor.

Hash functions (moderate weakening). Cryptographic hash functions such as SHA-256, RIPEMD-160, and HASH160 are affected by Grover's algorithm [9], which provides a quadratic speedup for unstructured search. This reduces the effective security of an n-bit hash from 2^n to $2^{n/2}$ for pre-image attacks, and from $2^{n/2}$ to $2^{n/3}$ for collision attacks. Importantly, this is a *weakening*, not a break: a 160-bit hash retains approximately 80 bits of pre-image resistance and approximately 53 bits of collision resistance against a quantum adversary. For our scheme, which relies on RIPEMD-160 (160 bits), this provides a comfortable security margin.

Practical timeline considerations. While the timeline for large-scale fault-tolerant quantum computers remains uncertain, there is broad consensus that breaking elliptic curve cryptography is likely to be feasible significantly earlier than breaking hash functions to a practically exploitable degree. Shor's algorithm requires a number of logical qubits roughly proportional to the key size, while Grover's algorithm—though polynomial in qubits—offers only a quadratic speedup and is difficult to parallelize. This asymmetry motivates our approach: we aim to eliminate *all* reliance on elliptic curve hardness for security, while retaining hash-based primitives whose quantum-reduced security levels remain adequate.

2.3.2 Security Requirements

In the context of quantum-safe transactions, there are two distinct security properties we aim to achieve. They differ significantly in severity and in the assumptions they make about the adversary.

1. Second pre-image resistance (critical). This is the primary security requirement. Suppose an honest spender broadcasts a transaction T whose digest is $D = H(T)$. We require that it is infeasible for a malicious actor—even one equipped with a quantum computer—who observes T and its digest D to construct a different transaction T' such that $H(T') = D$. If such a T' could be found, the attacker could reuse the Lamport signature revealed in T to authorize T', completely breaking the quantum safety of the scheme.

2. Collision resistance (secondary). Here we consider a potentially malicious spender who controls the transaction creation process. We require that the spender cannot find two distinct transactions T and T' such that $H(T) = H(T')$. This property is not essential for the spender's own quantum safety—the spender has no incentive to attack their own transaction—but it prevents equivocation: the spender cannot present one transaction to a counterparty while secretly holding another, equally valid transaction with a different set of outputs. This is particularly relevant for protocols such as BitVM [2] that rely on the uniqueness of the digest for off-chain dispute resolution.

2.4 Binohash Overview

Readers already familiar with Binohash *[2] may skip to Section 3, where we present our modifications.*

At a high level, to create a valid spending transaction with Binohash, the spender must complete two steps of proof-of-work:

Step 1: Transaction pinning. Force a W_1-bit PoW puzzle for every change to the spending transaction. This means the spender must perform 2^{W_1} additional work each time any part of the transaction is modified.

Step 2: Compute the Binohash digest. Find a valid hash digest D for the transaction. A valid digest is constructed from two rounds of "n-choose-t" subset selections over signatures embedded in the script—hence the name Binohash. The correct combination of signatures results in solving an additional W_2-bit PoW puzzle per round. The selected subsets form the digest D, which is also Lamport-signed within the script.

The Lamport signature uses a HORS (Hash to Obtain Random Subset) scheme [11]: for each of the n possible indices, the locking script contains a hash commitment $H(pre_i)$. To sign a particular subset (the digest), the spender reveals the preimages pre_i for the selected indices. The script verifies each preimage by hashing it and comparing to the commitment. Since preimages are secret until revealed, the spender commits to a specific digest at signing time.

The total work for the honest spender is therefore $2^{W_1} + 2 \cdot 2^{W_2}$. The digest D has $2W_2$ bits of entropy.

Second pre-image resistance. Both steps contribute equally to this property. Given a transaction T with $H(T) = D$, finding a different transaction T' with $H(T') = D$ costs 2^{W_1} per attempt (due to pinning), and the expected number of attempts to match D is 2^{2W_2} (the digest space). The total cost is $2^{W_1} \cdot 2^{2W_2}$.

Collision resistance and the role of pinning. The value of Step 1 (pinning) becomes apparent when we consider collision resistance. Due to the birthday attack, finding two transactions with the same digest costs only 2^{W_2} samples (the square root of the 2^{2W_2} digest space). However, generating each sample still requires 2^{W_1} work for pinning. The total collision cost is therefore $2^{W_1+W_2}$, and the W_1 contribution from pinning is the dominant factor.

Additionally, pinning is extremely cheap in terms of both opcodes and script size, while still providing valuable security. Why not increase W_1 further? Because the pinning cost is imposed on the honest spender as well. The optimal strategy is to set W_1 roughly equal to the work the spender already performs to find the digest, which is approximately $2 \cdot 2^{W_2}$. This balances the cost borne by the spender against the security gained.

2.5 Why a Binomial Structure?

The opcode constraint is extremely severe. Achieving a target security of ∼100 bits—with a digest that is also Lamport-signed—within a budget of 201 non-push opcodes is a formidable challenge.

The digest information must be provided by the unlocking script (the witness). If it were supplied as many independent elements, the cost of processing and signing each one would be prohibitive. Conversely, reproducing many elements in the locking script from a few witness-provided values is equally expensive: flow-control opcodes (`OP_IF`/`OP_ELSE`) count toward the 201-opcode limit even in non-executed branches, making dispatch logic impractical.

We are therefore forced to encode the digest in a *small number of witness elements*, and to verify and sign it with correspondingly few operations. This naturally leads to a binomial structure: the witness provides t indices selecting a subset from a pool of n items embedded in the locking script. The number of possible subsets, $\binom{n}{t}$, grows rapidly even for small t, providing the entropy we need. For example, $\binom{150}{8} \approx 2^{42}$ with just 8 selections.

2.6 Primitives

To perform proof of work over a transaction that is checkable in Script, we need two ingredients:

1. **A method to verify PoW in Script.** Given a candidate solution, the script must be able to check that sufficient work was performed. This is the role of the *signature size puzzle* (Section 2.6.1).

2. **A method to iterate over candidates.** The spender must be able to try many candidates until one satisfies the PoW check. This iteration itself splits into two distinct mechanisms:

   (a) **Iteration outside the script.** The spender modifies transaction parameters—such as `nLocktime`, output ordering, or `OP_RETURN` data—and re-checks the PoW puzzle in Script for each variant. Since these modifications happen outside the script, they are *not* Lamport-signable. This is essentially what is used for *transaction pinning* (Step 1): the spender iterates over transaction variants off-chain, and the script verifies the PoW for whichever variant is broadcast.

   (b) **Iteration inside the script.** Here, different iterations are invoked by different data provided in the unlocking script (witness). Crucially, this data *is* Lamport-signable within the script, which is what makes it suitable for the digest (Step 2). For this to be practical, we need an efficient method to iterate over many candidates with as few opcodes and bytes as possible. This is the role of the *FindAndDelete mechanism* (Section 2.6.2).

2.6.1 Signature Size Puzzle

The signature size puzzle, introduced by Bor [6] and refined by Linus [2], exploits the fact that a DER-encoded ECDSA signature's total byte length is determined by the lengths of its two components, r and s. If r is fixed, then the total signature size is governed solely by the length of s, and finding an s shorter than a target threshold requires brute-force search—a proof of work. The way to effectively fix r is by imposing an extreme constraint on the total signature size, practically forcing the spender to use the smallest known r value; any larger r would make it infeasible to meet the size target.

Concretely, the puzzle works as follows:

1. **Force a fixed r via size constraint.** The locking script enforces a signature size threshold so tight that only signatures using the smallest known r value—$r_{\min}$, whose x-coordinate is approximately 21 bytes—can satisfy it. The discrete logarithm of the corresponding curve point is known, allowing anyone to compute signatures with this specific r.

2. **Grind for small s.** To solve the puzzle, the spender searches over transactions (or nonces) until the resulting ECDSA signature—computed with the known $r_{\min}$—has an s value short enough that the total DER-encoded signature falls below the target size.

3. **Verify via `OP_SIZE`.** Bitcoin Script cannot parse the internal structure of a signature, but it can measure its total byte length using `OP_SIZE`. By checking that the signature is shorter than a threshold, the script implicitly verifies that sufficient grinding was performed.

This primitive is elegant and cheap: verification requires only `OP_SIZE` and an `OP_LESSTHAN` or `OP_EQUALVERIFY`—just 2–3 opcodes.

2.6.2 FindAndDelete and Sighash Variation

The signature size puzzle gives us a way to verify proof of work on a single signature. But for Step 2—computing the digest—we need to iterate over many candidates *inside* the script, in a way that is Lamport-signable. The mechanism that makes this possible is *FindAndDelete*—a legacy quirk of how `OP_CHECKMULTISIG` computes sighashes in pre-SegWit Bitcoin.

When `OP_CHECKMULTISIG` verifies a signature, it first computes the sighash—the double-SHA256 of the transaction data that the signature commits to. Crucially, the sighash is computed over the *scriptCode*, which is the locking script with all signature data removed via a process called FindAndDelete: before hashing, the script interpreter scans the scriptCode and deletes every occurrence of each signature being verified.

Binohash exploits this as follows. The locking script embeds n dummy signatures (e.g., $n = 150$). The spender selects a subset of t of them and passes them to `OP_CHECKMULTISIG`. FindAndDelete removes exactly those t signatures from the scriptCode before computing the sighash. A different subset of t signatures leads to a different scriptCode, and therefore a different sighash.

This gives the spender $\binom{n}{t}$ distinct sighashes to try—all from a single, fixed transaction. For each sighash, the spender checks whether the corresponding PoW puzzle is satisfied. The subset that solves the puzzle becomes the digest.

The dummy signatures are constructed to be transaction-independent using the `SIGHASH_SINGLE` bug: when `SIGHASH_SINGLE` is used for an input whose index exceeds the number of outputs, Bitcoin sets the signed message to $z = 1$, regardless of the transaction content [2]. This allows the dummy signatures and their corresponding public keys (recovered via ECDSA key recovery with $z = 1$) to be precomputed once and reused across all transactions.
FindAndDelete was removed in SegWit (BIP 143 [7]), which is one of the reasons this construction requires legacy (pre-SegWit) script execution.

2.7 The Binohash Scheme

We now describe how the primitives above are assembled into the full BINOHASH construction. For a complete treatment, see [2].

2.7.1 Script Verification (Locking Script)

Step 1: Transaction pinning. The locking script verifies four signature size puzzles, each using a different sighash flag combination, to cover all possible transaction modification vectors.[3] The four distinct puzzles can be enforced either via `OP_CODESEPARATOR`—which changes the scriptCode used for sighash computation at each invocation—or via a more compact trick involving carefully chosen public keys; see [2, Appendix A]. Additionally, the `SIGHASH_SINGLE` flag is explicitly excluded via a check, preventing an attacker from using the $z = 1$ bug to bypass the PoW. This step costs 13 opcodes.

Step 2: Digest verification (two rounds). Each round performs the following:

1. **Lamport signature verification.** The spender reveals HORS preimages for the selected subset indices. The script verifies each preimage against its corresponding hash commitment (embedded in the locking script).
2. **Dummy signature subset selection.** Based on the indices provided in the witness, the script uses `OP_ROLL` to select t dummy signatures from the pool of n.
3. **Multisig verification with puzzle signature.** The selected dummy signatures, together with the puzzle signature, are verified via `OP_CHECKMULTISIG`. FindAndDelete removes the selected dummies from the scriptCode, so the puzzle signature's sighash reflects the specific subset chosen.
4. **PoW check.** The puzzle signature's size is checked via `OP_SIZE` against the W_2 target.

2.7.2 Spending Procedure (Off-Chain)

To find a valid spending transaction, the spender performs the following:

1. **Solve the pinning puzzle.** Iterate over transaction parameters (e.g., `nLocktime`, output ordering) until all four pinning puzzles are satisfied. The transaction is now fixed.
2. **Search for the digest.** For each round, iterate over the $\binom{n}{t}$ possible dummy signature subsets. Each subset produces a different scriptCode via FindAndDelete, yielding a different sighash. Check whether the resulting puzzle signature satisfies the W_2-bit PoW target.
3. **Record the digest.** The subset that solves the puzzle becomes the digest for that round. Reveal the corresponding HORS preimages (Lamport signature).
4. **Repeat for both rounds.** If both rounds find a solution—the full digest is complete. If either round fails (none of the $\binom{n}{t}$ subsets solve the puzzle), return to step 1 with a modified transaction and try again.

[3] Since Bitcoin Script cannot enforce which sighash flag the spender uses, the spender could choose any of the six available modes. Two of these are `SIGHASH_SINGLE` variants, which are eliminated by an explicit check that detects the $z = 1$ bug. The remaining four modes—`ALL`, `NONE`, `ALL|ANYONECANPAY`, and `NONE|ANYONECANPAY`—are each assigned a separate puzzle. By requiring all four puzzles to be solved, we ensure that regardless of which mode the spender uses, every transaction field is committed to by at least one puzzle. See [2, Section 5] for details.

2.8 Open Issues

Before presenting our scheme, we identify two issues with the BINOHASH construction as described above. The first is a fundamental quantum safety vulnerability—a direct consequence of the catastrophic weakness of elliptic curve cryptography under quantum computing (Section 2.3.1). The second is a sighash-related attack that, while not specific to quantum computing, must be addressed for standalone use outside of BitVM.

2.8.1 The Signature Size Puzzle Is Not Quantum Safe

The signature size puzzle (Section 2.6.1) relies on the assumption that the spender is forced to use the smallest known r value, $r_{\min}$, when constructing the puzzle signature. This assumption holds classically: finding a different curve point with a comparably small x-coordinate requires approximately 2^{96} work [2, Section 4].

As discussed in Section 2.3.1, a quantum computer running Shor's algorithm [8] can compute the discrete logarithm of *any* elliptic curve point in polynomial time. This includes points with very small x-coordinates—in particular, the point with $r = 1$ (or the smallest valid r on the curve). Armed with this value, the adversary can produce signatures with a shorter r than $r_{\min}$, allowing them to satisfy the `OP_SIZE` check with significantly less work on s—or none at all.

Concretely: the PoW difficulty is determined by how short s must be to bring the total signature size under the target threshold. If the target requires W bits of grinding on s when using $r_{\min}$, then an adversary who finds a curve point with an r value just $\lceil W/8 \rceil$ bytes shorter than $r_{\min}$ gains enough headroom to satisfy the size check with virtually any s, reducing the PoW to zero. For typical targets of $W \approx 40$, this means finding an r just 5 bytes shorter than $r_{\min}$ suffices to break the puzzle entirely.[4]

Note that this vulnerability is fundamentally different from the weakening of hash functions by Grover's algorithm. While Grover reduces hash security by a square root factor, the result remains computationally hard (e.g., RIPEMD-160 retains $\sim 2^{80}$ pre-image resistance). Shor's attack on the signature size puzzle, by contrast, is a *total break*: the PoW guarantee collapses entirely, regardless of the target difficulty.

2.8.2 Sighash Flag Uncertainty

The second issue concerns the sighash flag byte appended to each signature. As discussed in Section 2.7, the pinning stage uses four separate puzzles to cover all non-`SINGLE` sighash modes. However, this protection applies only to the pinning signatures.

The puzzle signatures in Step 2 (the digest rounds) are produced by the spender via PoW grinding. The spender is free to choose any sighash flag for these signatures. In particular, they could use `ANYONECANPAY|NONE`, which commits to neither the outputs nor the other inputs of the transaction. As Linus observes [2, Section 7.1], this allows an attacker to take a valid puzzle signature—which committed to a specific set of outputs—and reuse it in a completely different transaction with different outputs, provided the input being spent remains the same.

In the BitVM context, this attack is mitigated by off-chain verification: the BitVM protocol can check the sighash mode and reject non-compliant transactions [2, Section 7.1]. However, for standalone use—where we want the on-chain script alone to guarantee transaction integrity—this mitigation is insufficient. The script must enforce the sighash flag directly, or the construction must be redesigned so that the sighash flag is inherently controlled.

[4] As noted in Section 1.1.1, once the discrete logarithm for the smallest valid r is known, the puzzle *could* be redesigned around it. But all existing transactions created using $r_{\min}$ would remain vulnerable.

3 Our Scheme

We now present our modifications to BINOHASH. The central change is replacing the signature size puzzle (Section 2.6.1) with a *hash-based* puzzle whose security relies only on the pre-image resistance of cryptographic hash functions—not on any elliptic curve hardness assumption. This directly addresses the quantum vulnerability identified in Section 2.8.1.

3.1 Hash-to-Signature Puzzle

3.1.1 Core Observation

The replacement puzzle is based on the following observation, first noted by Linus [10]:

Observation. *The output of a cryptographic hash function can, with some probability, be a valid DER-encoded ECDSA signature.*

A DER-encoded ECDSA signature has a rigid structure: `30 [len] 02 [r-len] [r] 02 [s-len] [s] [sighash]`, where the length fields must be internally consistent, both r and s must be positive integers with no unnecessary leading zeros (i.e., MSB < 128), and the sighash byte must be a valid flag. A random byte string satisfying all of these constraints simultaneously is a rare event—but not impossibly rare.

This gives us a new type of PoW puzzle: *grind for an input whose hash output happens to be a valid DER signature.* The work required is determined by the probability that a random output of the hash function satisfies the DER constraints.

3.1.2 SHA-256 Variant ($\sim 2^{51}$ work)

Linus [10] first described this approach using SHA-256. A random 32-byte string is a valid DER-encoded ECDSA signature with probability approximately 2^{-51}. The DER structure admits multiple valid combinations of r and s lengths within 32 bytes; the dominant constraint is that both r and s must have MSB < 128 (each contributing roughly 1 bit of rejection), combined with the requirement that the length fields are internally consistent.

Verification in Script is straightforward: compute `OP_SHA256` on the input, then use `OP_CHECKSIG` (or `OP_CHECKMULTISIG`) to verify the result as a signature. If the hash output is not valid DER, the signature check fails.

3.1.3 RIPEMD-160 Variant ($\sim 2^{46}$ work)

As noted in BINOHASH Appendix D [2], the same approach works with RIPEMD-160, producing 20-byte outputs. A random 20-byte string is a valid DER-encoded ECDSA signature with probability approximately 2^{-46}.

The higher probability arises because 20-byte DER signatures, while shorter, admit a wider range of valid (r-len, s-len) combinations relative to their total length. The dominant constraints remain the same: the `30` tag byte, consistent length fields, positive r and s values with MSB $<$ 128, and a valid sighash byte. See [2, Appendix D] for a detailed enumeration of the valid DER formats at 20 bytes.

Verification in Script uses `OP_RIPEMD160` instead of `OP_SHA256`. The lower work requirement (2^{46} vs. 2^{51}) makes the RIPEMD-160 variant more practical, and we adopt it as the default for our construction.

3.1.4 Quantum Safety

Unlike the signature size puzzle, the hash-to-signature puzzle does not depend on any elliptic curve hardness assumption. The work required is determined entirely by the pre-image resistance

of the hash function: the spender must find an input whose hash output satisfies the DER structural constraints. A quantum adversary gains only Grover's quadratic speedup [9]:

- RIPEMD-160: 2^{46} classically $\rightarrow \sim 2^{23}$ with Grover (on the puzzle alone).
- SHA-256: 2^{51} classically $\rightarrow \sim 2^{25.5}$ with Grover.

3.1.5 ECDSA Key Recovery

A standard property of ECDSA—used extensively in both BINOHASH and our scheme—is that given a valid signature (r, s) and the signed message hash z, one can *recover* the public key that produced the signature, without knowing the private key. This is known as ECDSA public key recovery.

The recovery works as follows. The signature component r is the x-coordinate of a curve point $R = k \cdot G$ (where k is the ephemeral nonce). Given r, we can reconstruct the candidate point R (up to a small number of possibilities, typically 2, corresponding to the two y-coordinates for a given x). From the ECDSA verification equation:

$$s = k^{-1}(z + r \cdot d) \pmod{n}$$

we can solve for the public key $Q = d \cdot G$:

$$Q = r^{-1}(s \cdot R - z \cdot G)$$

This computation involves only public values—r, s, z, and the curve parameters—and can be performed by anyone. No private key or secret information is required.

Crucially, key recovery is *not* a security assumption—it is a public computation that anyone can perform, including a quantum adversary.

3.2 Using the Hash-to-Signature Puzzle in Script

We now describe how the hash-to-signature puzzle and ECDSA key recovery are combined to create a quantum-safe PoW check that is fully verifiable in Bitcoin Script.

Step 1: Hardcoded signature with a fixed sighash flag. The locking script contains a hardcoded signature sig_{nonce} with a predetermined sighash flag—in our case, `SIGHASH_ALL`. Because the signature is embedded directly in the script (rather than provided by the spender), the sighash flag is fixed at script creation time. This immediately addresses the sighash flag uncertainty identified in Section 2.8.2: the spender cannot choose a different flag.

Step 2: Derive and verify the transaction-bound key. Given sig_{nonce} and the spending transaction, there is a deterministic function—ECDSA key recovery (Section 3.1.5)—that produces the corresponding public key:

$$key_{\text{nonce}} = \text{Recover}(sig_{\text{nonce}}, z)$$

where z is the sighash of the transaction. The spender computes key_{nonce} off-chain and provides it in the witness. The script verifies the binding via `OP_CHECKSIGVERIFY`: if key_{nonce} does not match (sig_{nonce}, z), verification fails. Since z depends on the transaction content, key_{nonce} is uniquely determined by the transaction—changing any part of the transaction changes z, and therefore changes key_{nonce}.

Step 3: Hash-to-signature puzzle check. The script computes $sig_{\text{puzzle}} = \texttt{OP_RIPEMD160}(key_{\text{nonce}})$. For the puzzle to be solved, this 20-byte output must be a valid DER-encoded ECDSA signature—an event with probability $\sim 2^{-46}$. To verify this in Script, the spender provides a second public key key_{puzzle} (derived via ECDSA key recovery from $(sig_{\text{puzzle}}, z')$, where z' is the relevant sighash). The script verifies $(sig_{\text{puzzle}}, key_{\text{puzzle}})$ via `OP_CHECKSIGVERIFY`: if sig_{puzzle} is not valid DER, this check fails.

Result. If both verifications pass, the puzzle is solved. The chain of trust is:

1. sig_{nonce} is hardcoded $\rightarrow$ its sighash flag is fixed (`SIGHASH_ALL`).
2. $key_{\text{nonce}} = \text{Recover}(sig_{\text{nonce}}, z) \rightarrow$ bound to this specific transaction.
3. $sig_{\text{puzzle}} = \text{RIPEMD-160}(key_{\text{nonce}}) \rightarrow$ inherits the transaction binding.
4. key_{puzzle} verifies that sig_{puzzle} is valid DER $\rightarrow$ puzzle check complete.

Fixed puzzle difficulty. An important distinction between the hash-to-signature puzzle and the signature size puzzle is that the hash-to-signature puzzle has a *fixed* difficulty: it is determined entirely by the hash function and the target primitive (DER-encoded ECDSA signature). For RIPEMD-160, this is $\sim 2^{46}$; for SHA-256, $\sim 2^{51}$. There is no knob to tune.

By contrast, the signature size puzzle allows flexible difficulty: the script creator can choose any size threshold, adjusting the PoW target continuously. This makes script planning more constrained in our case: the entropy generated by the binomial subset selection (i.e., $\binom{n}{t}$) must be matched to the fixed 2^{46} or 2^{51} puzzle target. A mismatch—where $\binom{n}{t}$ is significantly smaller or larger than the puzzle target—results in either excessive transaction grinding (if too small) or wasted entropy (if too large). We analyze the impact on off-chain work in Section 4.

3.3 Full Scheme

We now describe the complete construction, replacing the signature size puzzle in Binohash with the hash-to-signature puzzle throughout.

3.3.1 Transaction Pinning

Transaction pinning ensures that modifying any part of the spending transaction requires solving the hash-to-signature puzzle anew ($\sim 2^{46}$ work). In Binohash, this requires four signature size puzzles at a cost of 13 opcodes. In our scheme, a single application of the hash-to-signature puzzle suffices, since the hardcoded `SIGHASH_ALL` flag commits to all transaction fields.

The pinning script is a direct instantiation of the puzzle described in Section 3.2:

```
// Witness: <key_puzzle> <key_nonce>       (key_nonce on top)

<sig_nonce>                              // hardcoded, SIGHASH_ALL (data
   push)
OP_OVER                            // [1] copy key_nonce
OP_CHECKSIGVERIFY                  // [2] verify (sig_nonce, key_nonce)
OP_RIPEMD160                       // [3] key_nonce -> sig_puzzle
OP_SWAP                            // [4] get key_puzzle on top
OP_CHECKSIGVERIFY                  // [5] verify (sig_puzzle, key_puzzle)
```

Matching this to the steps in Section 3.2:

- **Step 1** (fixed sighash): `<sig_nonce>` is hardcoded with `SIGHASH_ALL`—the flag cannot be changed by the spender.

- **Step 2** (derive and verify key): `OP_OVER` copies `key_nonce` (so it survives for use in Step 3), then `OP_CHECKSIGVERIFY` verifies that `key_nonce` matches `(sig_nonce, sighash)`. This binds `key_nonce` to this specific transaction.
- **Step 3** (hash-to-signature check): `OP_RIPEMD160` hashes `key_nonce` to produce `sig_puzzle`. Then `OP_SWAP` brings `key_puzzle` on top, and the second `OP_CHECKSIGVERIFY` verifies that `sig_puzzle` is a valid DER signature.

Total cost: **5 opcodes**—a saving of 8 compared to BINOHASH's 13-opcode pinning. The pinning is self-contained: after both checks, nothing remains on the stack.

3.3.2 Digest Rounds

The puzzle signature is verified inside `CHECKMULTISIG`, which means FindAndDelete modifies the scriptCode—and therefore the sighash—depending on which dummy signatures are included. This gives us $\binom{n}{t}$ distinct sighash candidates from a single fixed transaction, each corresponding to a different subset of dummy signatures. The purpose of the selection stage is to translate the digest provided in the witness into the specific subset that solves the puzzle, after verifying the Lamport signature on it.

Each digest round follows the same high-level structure as BINOHASH (Section 2.7) and proceeds in three stages:

Stage 1: Subset selection and Lamport verification. This stage is identical to BINOHASH. The spender provides t indices in the witness, each selecting one dummy signature from the pool of n. For each selected index, the script verifies the corresponding HORS preimage against its hash commitment—this constitutes the Lamport signature over the digest. The selected dummy signatures are left on the stack for use in Stage 3. The detailed script for this stage is given in Appendix A.

Stage 2: Puzzle signature derivation. The spender provides key_{nonce} in the witness. The script duplicates it and computes $sig_{\text{puzzle}} = \texttt{OP_RIPEMD160}(key_{\text{nonce}})$. One copy of key_{nonce} is consumed by the hash; the other is preserved for use as a public key in Stage 3. The RIPEMD-160 output sig_{puzzle} is left on top of the stack as the puzzle signature.

Stage 3: Verification. First, the script verifies that sig_{puzzle} is a valid DER signature by rolling key_{puzzle} from the witness and executing `OP_CHECKSIGVERIFY` on the pair $(sig_{\text{puzzle}}, key_{\text{puzzle}})$. This check must happen *before* `CHECKMULTISIG`—not inside it—because placing sig_{puzzle} inside `CHECKMULTISIG` would create a circular dependency: the sighash determines key_{nonce}, which determines sig_{puzzle}, which would in turn affect the sighash.

After sig_{puzzle} is consumed by `CHECKSIGVERIFY`, the remaining key_{nonce} copy is on the stack above the dummy signatures. The script then executes a $(t+1)$-of-$(t+1)$ `CHECKMULTISIG`, verifying the t selected dummy signatures and sig_{nonce} (hardcoded, with `SIGHASH_ALL`—resolving the sighash flag vulnerability of Section 2.8.2) against their corresponding public keys. key_{nonce} serves as the public key for sig_{nonce}.

The critical point is that sig_{nonce} goes through `CHECKMULTISIG`, where FindAndDelete modifies the scriptCode before computing sighashes. The sighash that sig_{nonce} is verified against therefore depends on *which* dummy signatures were selected. This means $key_{\text{nonce}} = \text{Recover}(sig_{\text{nonce}}, z)$ is different for each subset, and consequently $sig_{\text{puzzle}} = \text{RIPEMD-160}(key_{\text{nonce}})$ is different as well. The subset that happens to produce a valid DER output becomes the digest for this round.

The script for Stages 1–3:

```
// Witness (bottom to top):
//   <key_puzzle> <key_nonce>
//   <pub_t> ... <pub_1>
//   <preimage_t> ... <preimage_1>
//   <index_t> ... <index_1>

// Locking script:
<H(pre_n)> ... <H(pre_1)>     // n HORS commitments
<sig_n> ... <sig_1>            // n dummy sigs (SIGHASH_SINGLE)
OP_0                           // CHECKMULTISIG dummy
<sig_nonce>                      // hardcoded (SIGHASH_ALL)

// Stage 1: selection + Lamport verification
...                            // (see Appendix A)
// After Stage 1: selected dummy sigs + sig_nonce + OP_0 on stack

// Stage 2: derive puzzle sig
{pos} OP_ROLL              // roll key_nonce from witness
OP_DUP                     // copy (one for RIPEMD, one for pubkey)
OP_RIPEMD160               // key_nonce -> sig_puzzle

// Stage 3a: verify sig_puzzle is valid DER
{pos} OP_ROLL              // roll key_puzzle from witness
OP_CHECKSIGVERIFY          // verify (sig_puzzle, key_puzzle)

// Stage 3b: CHECKMULTISIG (t+1)-of-(t+1)
OP_{t+1}                   // M = t+1 signatures
OP_2 OP_ROLL               // move key_nonce copy into pubkey zone
{pos} OP_ROLL (x t)        // roll t dummy pubkeys from witness
OP_{t+1}                   // N = t+1 public keys
OP_CHECKMULTISIG           // verify t dummies + sig_nonce
```

Here key_{puzzle} is derived by the spender via ECDSA key recovery from (sig_{puzzle}, z)—the same mechanism used throughout the scheme.

The per-round overhead compared to BINOHASH is +4 opcodes for the puzzle and `CHECKMULTISIG` setup. Combined with the 8-opcode saving from pinning (Section 3.3.1), the net overhead for a two-round scheme is **exactly zero**.

Two rounds. The full scheme uses two independent rounds, each with its own pool of n dummy signatures, HORS commitments, and hardcoded sig_{nonce}. The full digest is the concatenation of the two rounds' selected subsets. The second round's script is identical in structure to the first; only the embedded data differs.

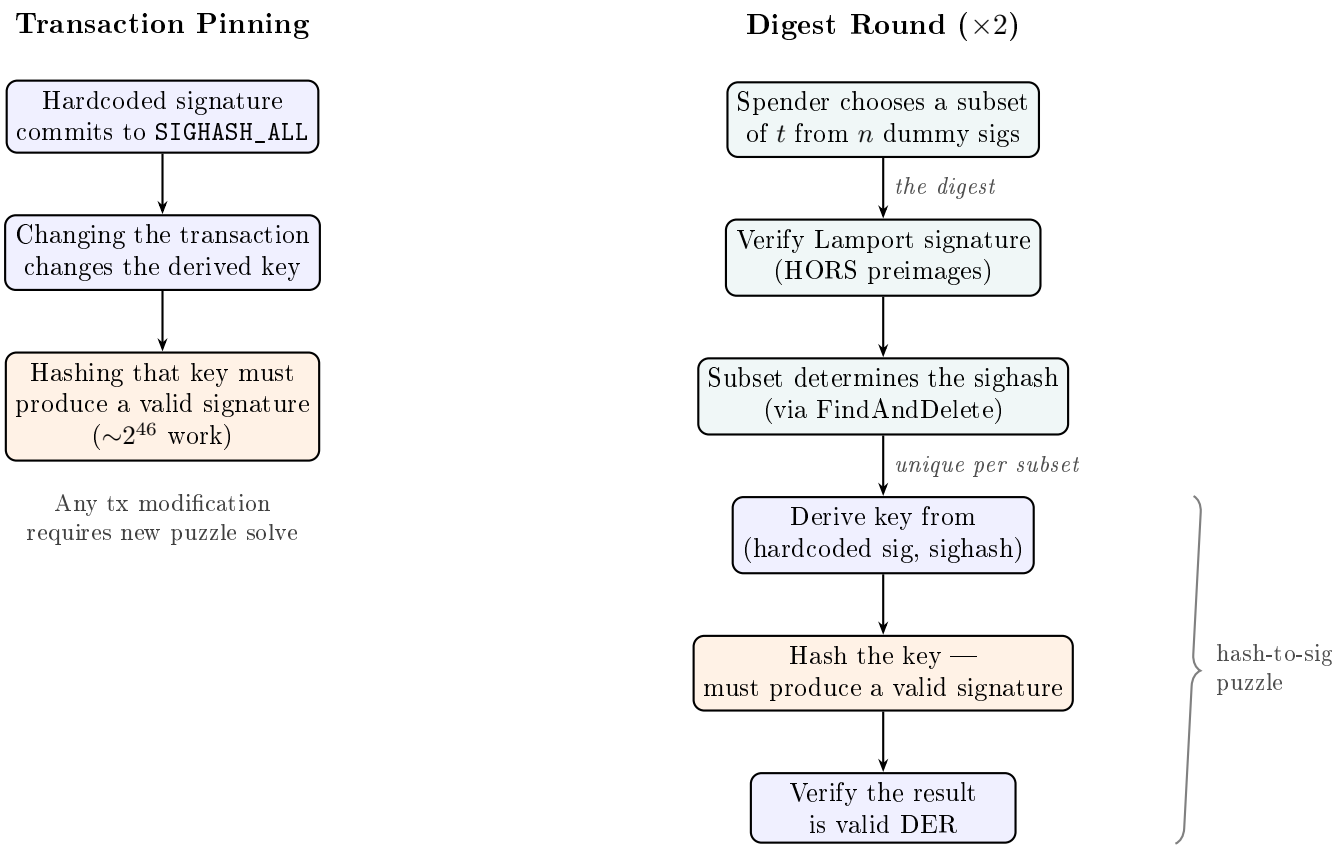

Figure 1: The quantum-safe BINOHASH scheme. **Left:** transaction pinning ensures each transaction attempt costs $\sim 2^{46}$ work. **Right:** each digest round selects a subset (the digest), verifies its Lamport signature, then checks the hash-to-signature puzzle—the subset that produces a valid RIPEMD-160 output is the solution. Two independent rounds are executed.

Bonus Keys. [TODO]

4 Security and Cost Analysis

We analyze the security of our scheme for the baseline configuration: $n = 150$, $t = 8$ signed selections per round, two rounds. The puzzle target is fixed at $\sim 2^{46}$ (RIPEMD-160).

4.1 Baseline: $t = 8$, $t = 8$

Property	**Value**
Non-push opcodes	197 / 201 (4 spare)
Script size	∼9,650 / 10,000 bytes
Subsets per round	$\binom{150}{8} \approx 2^{42.3}$
Puzzle target per round	$\sim 2^{46}$ (RIPEMD-160)
Digest size	$2 \times 42.3 = 84.5$ bits
Tx grinds	$\left(2^{46}/2^{42.3}\right)^2 \approx 2^{7.5}$ (≈180 attempts)
Honest spender work	$\sim 2^{53.5}$
Second pre-image resistance	$\sim 2^{138}$
Collision resistance	$\sim 2^{88}$

4.2 Opcode and Script Size Budget

Opcodes. The pinning stage costs 5 non-push opcodes. Each digest round costs:

$$11t + 8$$

where t is the number of selections per round. The breakdown is: $9t$ for the selection loop (9 opcodes per selection) and $2t+8$ for the puzzle derivation, `CHECKSIGVERIFY`, and `CHECKMULTISIG` setup. For two rounds with t_1 and t_2 selections, the total is:

$$5 + (11t_1 + 8) + (11t_2 + 8) = 11(t_1 + t_2) + 21 \leq 201$$

Script size. The pinning stage occupies approximately 15 bytes (a 9-byte signature push plus 5 opcode bytes). Each digest round occupies approximately:

$$31n + 13t + 20 \text{ bytes}$$

The dominant term is $31n$: each of the n pool entries contributes a 20-byte HORS commitment (plus 1-byte push opcode) and a 9-byte dummy signature (plus 1-byte push opcode). The remaining terms account for the selection logic, puzzle verification, and `CHECKMULTISIG` setup. For two rounds, the total is approximately $62n + \text{overhead} \leq 10{,}000$.

At $n = 150$, each additional pool entry costs $\sim$62 bytes across both rounds, while adding $\sim$0.08 bits of digest entropy per entry.

4.3 Security and Grinding Analysis

Second pre-image resistance. An attacker who observes a valid spending transaction must find a different transaction that produces the same digest—the same signed indices in both rounds. Each transaction attempt costs 2^{46} (the pinning puzzle). For each pinned transaction, the probability that a specific t-subset solves the round puzzle is 2^{-46}. Since both rounds must independently match, the probability per pinned attempt is 2^{-92}. The total cost is therefore $2^{46} \times 2^{92} = 2^{138}$.

Collision resistance. A malicious spender who controls the transaction creation process aims to find two transactions with the same digest. The digest space is $2^{84.5}$. By the birthday bound, $\sim 2^{42.3}$ samples suffice to find a collision. Each sample costs 2^{46} (pinning), giving a total collision cost of $2^{46+42.3} \approx 2^{88}$.

Honest spender: subset mismatch. The subsets per round ($\binom{150}{8} \approx 2^{42.3}$) fall short of the 2^{46} puzzle target by a factor of $2^{3.7}$. This means each round has only a $2^{-3.7}$ probability of containing a valid subset for a given pinned transaction. Since both rounds must succeed, the spender needs on average $\sim 2^{7.5} \approx 180$ pinned transaction attempts. Each attempt requires iterating over all $\binom{150}{8}$ subsets per round, giving a total work of $\sim 2^{53.5}$ RIPEMD-160 evaluations. The actual wall-clock cost depends on implementation; we defer this estimate to Section 4.5.

4.4 Quantum Security

We analyze the security of the baseline configuration under quantum adversaries, distinguishing between two threat levels introduced in Section 2.3.1.

Shor's algorithm only (ECDSA broken, hash functions unaffected). This is the primary threat our scheme is designed to resist. A quantum adversary running Shor's algorithm can compute discrete logarithms and forge ECDSA signatures, but cannot speed up hash function evaluation.

Under this model, the security of our scheme is *identical* to the classical case:

Property	**Classical**	**Shor only**
Pinning cost	2^{46}	2^{46} (hash-based)
Second pre-image	2^{138}	2^{138}
Collision	2^{88}	2^{88}

This is the central result: because our puzzle relies on RIPEMD-160 pre-image resistance rather than ECDSA hardness, Shor's algorithm provides *no advantage* to the attacker. The scheme achieves full classical security even against a quantum adversary with unbounded access to Shor's algorithm.

Grover's algorithm (quadratic speedup on hash search). Grover's algorithm [9] provides a quadratic speedup for unstructured search problems. Applied to our scheme, it reduces the effective security:

Property	**Classical**	**With Grover**
Pinning cost	2^{46}	$\sim 2^{23}$
Second pre-image	2^{138}	$\sim 2^{69}$
Collision	2^{88}	$\sim 2^{65}$

For second pre-image resistance, Grover provides a square-root speedup on the full search: the classical cost of 2^{138} becomes $\sim 2^{69}$. For collision resistance, the pinning cost per sample drops from 2^{46} to $\sim 2^{23}$, while the birthday bound remains at $2^{42.3}$ samples, giving $2^{23+42.3} \approx 2^{65}$.

Practical feasibility of Grover. While these reductions appear significant on paper, Grover's algorithm faces fundamental limitations that make it far less threatening than Shor's in practice. Grover cannot be efficiently parallelized: running k quantum computers in parallel provides only a $\sqrt{k}$ speedup, not a k-fold speedup [9]. Moreover, implementing the search oracle as a quantum circuit requires all intermediate state—including the sighash computation (SHA-256d) and key recovery—to be represented in reversible logic, demanding significant qubit overhead.

More importantly, Shor's algorithm for the secp256k1 curve requires on the order of a few thousand logical qubits and is expected to be feasible well before Grover becomes practical against targets of 2^{46} or higher. This asymmetry is precisely why eliminating reliance on ECDSA hardness—while retaining hash-based security—is the right design strategy.

[TODO: Bonus keys, comparison table, off-chain cost]

4.5 Off-Chain Cost

[TODO]

5 Optimizations

[TODO]

6 Conclusion

[TODO]

References

[1] E. Heilman, "Signing a Bitcoin Transaction with Lamport Signatures (no OP_CAT)," Bitcoin Development Mailing List, 2024. https://groups.google.com/g/bitcoindev/c/mR53go5gHIk

[2] R. Linus, "Binohash: Transaction Introspection Without Softforks," 2025. https://robinlinus.com/binohash.pdf

[3] E. Heilman, V. Kolobov, A. Levy, and A. Poelstra, "ColliderScript: Covenants in Bitcoin via 160-bit Hash Collisions," Cryptology ePrint Archive, Report 2024/1802, 2024. https://eprint.iacr.org/2024/1802

[4] P. Wuille, J. Nick, and T. Ruffing, "BIP 341: Taproot: SegWit version 1 spending rules," Bitcoin Improvement Proposals, 2020. https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki

[5] Marathon Digital Holdings, "Marathon Digital Holdings Launches Slipstream," Press Release, February 2024.

[6] A. Bor, "BTC PoW Locked Outputs," GitHub, 2024. https://github.com/adambor/btc-pow-locked-outputs

[7] J. Lau and E. Lombrozo, "BIP 143: Transaction Signature Verification for Version 0 Witness Program," Bitcoin Improvement Proposals, 2016. https://github.com/bitcoin/bips/blob/master/bip-0143.mediawiki

[8] P. W. Shor, "Algorithms for Quantum Computation: Discrete Logarithms and Factoring," Proceedings of the 35th Annual Symposium on Foundations of Computer Science, pp. 124–134, 1994.

[9] L. K. Grover, "A Fast Quantum Mechanical Algorithm for Database Search," Proceedings of the 28th Annual ACM Symposium on Theory of Computing, pp. 212–219, 1996.

[10] R. Linus, "SHA-2 ECDSA," GitHub, 2024. https://github.com/RobinLinus/sha2-ecdsa

[11] L. Reyzin and N. Reyzin, "Better than BiBa: Short One-Time Signatures with Fast Signing and Verifying," Proceedings of ACISP, pp. 144–153, 2002.

A Subset Selection Script

The following script implements a single signed selection iteration (Stage 1 of a digest round, Section 3.3.2). It is repeated t_{signed} times, with the position constants adjusted for each iteration i:

```
// Signed selection i (9 opcodes):
{2n+1-i} OP_ROLL            // roll index from witness
{n-i} OP_MIN                 // sanitize (prevent out-of-bounds)
OP_DUP                       // copy index (needed for sig lookup)
{n+1} OP_ADD                 // compute commitment position
OP_ROLL                      // roll HORS commitment to top
{2n+1+t-2i} OP_ROLL          // roll HORS preimage from witness
```

```
OP_HASH160                      // hash preimage
OP_EQUALVERIFY                  // verify preimage matches commitment
OP_ROLL                         // roll dummy sig (using index on stack)
```

This is identical to the selection loop in BINOHASH [2, Appendix B]. The position constants account for sig_{nonce} being on the stack above the dummy signatures (hence $2n + 1$ rather than $2n$).

B Binohash Scheme Diagrams

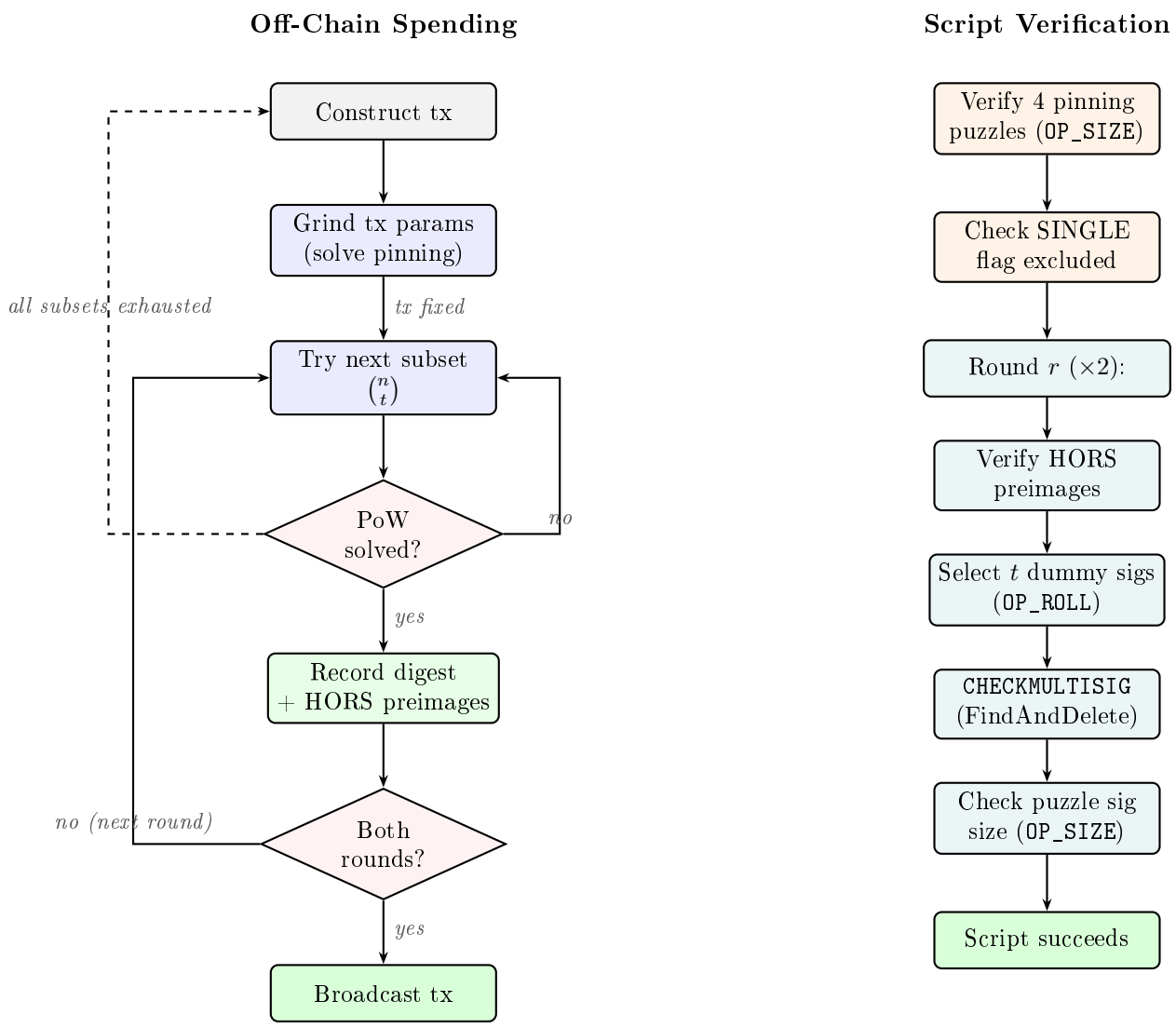

Figure 2: The BINOHASH construction. **Left:** the off-chain spending procedure. **Right:** the on-chain script verification.

C Full QSB Script ($t = 8$, $t = 8$, $n = 150$)

The complete locking script for the baseline configuration (197 opcodes, ∼9,650 bytes). HORS commitments and dummy signatures are abbreviated.

```
// ============================================================
// PINNING (5 opcodes)
// ============================================================
// Witness: <key_puzzle> <key_nonce>

<sig_nonce>                     // 9 bytes, hardcoded, SIGHASH_ALL
OP_OVER                         // [1] copy key_nonce
OP_CHECKSIGVERIFY               // [2] verify (sig_nonce, key_nonce)
OP_RIPEMD160                    // [3] key_nonce -> sig_puzzle
OP_SWAP                         // [4] get key_puzzle on top
OP_CHECKSIGVERIFY               // [5] verify (sig_puzzle, key_puzzle)


// ============================================================
// ROUND 1 (96 opcodes)
// ============================================================

// --- 150 HORS commitments (20 bytes each) ---
<H(pre_149)_r1>
<H(pre_148)_r1>
...
<H(pre_000)_r1>

// --- 150 dummy sigs (9 bytes each, SIGHASH_SINGLE) ---
<sig_149_r1>
<sig_148_r1>
...
<sig_000_r1>

OP_0                            // CHECKMULTISIG dummy
<sig_nonce_r1>                  // 9 bytes, hardcoded, SIGHASH_ALL

// --- 8 signed selections (9 ops each = 72 ops) ---
// Selection 0:
301 OP_ROLL                     // roll index_0 from witness
150 OP_MIN                      // sanitize
OP_DUP                          // copy index
151 OP_ADD                      // commitment position
OP_ROLL                         // roll commitment
309 OP_ROLL                     // roll preimage_0 from witness
OP_HASH160                      // hash preimage
OP_EQUALVERIFY                  // verify HORS
OP_ROLL                         // roll dummy sig
// Selections 1--7: (same structure, adjusted positions)
...

// --- Puzzle + CHECKMULTISIG (24 ops) ---
302 OP_ROLL                     // roll key_nonce from witness
OP_DUP                          // copy for RIPEMD + pubkey
OP_RIPEMD160                    // key_nonce -> sig_puzzle

302 OP_ROLL                     // roll key_puzzle from witness
OP_CHECKSIGVERIFY               // verify (sig_puzzle, key_puzzle)

OP_9                            // M = 9 (8 dummies + sig_nonce)
OP_2 OP_ROLL                    // move key_nonce into pubkey zone
302 OP_ROLL                     // roll pub_0
```

```
...                                 // (8 rolls total)
302 OP_ROLL                         // roll pub_7
OP_9                                // N = 9
OP_CHECKMULTISIG                    // verify all 9 sig/key pairs


// ============================================================
// ROUND 2 (96 opcodes) -- same structure as Round 1
// ============================================================
// (identical structure, independent data)
...
```

Full annotated scripts with exact position constants for multiple configurations are available at: https://github.com/avihu28/quantum-safe-bitcoin-tx